

Today's topics

Revise JSP Basics

JSP Directives

JSP Actions

JSP-JavaBean-DAO(using DBUtils)-Entities - DB

JSTL

page vs pageContext vs pageScope

page - JSP implicit object (accessible from scriptlets n expressions)

Represents translated page class instance !

eg : For index.jsp -> index_jsp - instance of this class

pageContext - instance of PageContext

Holds (HAS-A) - other implicit objects'

eg - getSession , getRequest , getResponse....

+

map of page scoped attributes : pageScope

page n pageContext -- from scriptlets n expression

pageScope - accessbile via EL syntax only !

1. Name JSP Scripting elements

scriptlets,expressions OR EL syntax, declarations

2. Name JSP implicit objects available from scriptlets n expression

request,response,out,session,config,application,exception,page,pageContext

3. Name EL implicit objects available from EL syntax(\${...})

param, pageScope,requestScope,sessionScope,applicationScope,cookie,initParams

4. Solve

How will you get the value of a cookie by the name "abc" , using EL syntax

`${cookie.abc.value}`

Java - `cookie.get("abc").getValue()`

5. Solve : what will be o/p

`${pageContext.session.maxInactiveInterval}`

Java - `pageContext.getSession().getMaxInactiveInterval()` - 30*60 -1800

6. Suppose : user_info is a session scoped attribute

What will this do ?

`${sessionScope.user_info}`

Print user info on the client browser

Java - `session.getAttribute("user_info").toString()` -> sent to client

7. Solve

`${pageContext.session.id}`

Java - `pageContext.getSession().getId()` -> sent to client

8. `${param.email}`

Java - `request.getParameter("email")` -> sent to client

9. How to invalidate session ?

Options :

1. Scriptlet :

`<%`

`session.invalidate();`

`%>`

2. JSP expr : `<%= session.invalidate() %>`

3. EL syntax : `${pageContext.session.invalidate()}`

Choose correct options : 1,3

2. Identify JSP Directives n important attributes

page

include

3. JSP Actions

3.1 Request dispatcher related

3.2 JSP using JavaBean related standard actions

Layers - JSP - Java Bean - JDBC based DAO (dependency - DB connection - provider - DBUtils) -

Entities|Model|POJOs - RDBMS

Servlet - to manage DB connection(openConnection n closeConnection)

Objective : User login in online voting

We have already solved it using servlets . Right ?

So why JSP Using Java Beans

For separation of concerns(=tasks/responsibilities)

We are going to separate :

1. P.L (presentation logic) , navigation in a JSP
(w/o using java code)

2. B.L (Business logic) : separated in a java class : java bean

Why Java beans ?

1. Separation between business logic n managing DAOs - java bean class
n presentation logic(JSP)

2. re usability

3. WC can automatically parse request parameters (string) -> primitive types.

What is a java bean ?

Java class containing B.L (equivalent to service layer in spring)

Server side attribute , created by WC , to be added under required scope
(page | request | session | application)

Start case study : Voting case study (admin role)

Layers to be copied from earlier (servlet based) assgnment

No change in : Tables, POJOs, DAOs , DBUtils

3. JSP standard Actions for Using java beans

3.1 Create Java bean

```
<jsp:useBean id="bean ID" class="Fully qualified java bean class name"
scope="page|request|session|application"/>
```

Default scope=page

eg - <jsp:useBean id="user_bean" class="com.app.beans.UserBean" scope="session"/>

Internals (What will WC invoke?)

refer to a digram

WC - session.getAttribute("user_bean") - for the 1st time -

rets null => bean doesn't exist

WC creates it n adds it under the specified scope

```
session.setAttribute("user_bean",new UserBean());
```

2. Data binding from view layer ---> model (java bean)

i.e calling the setters of Javabeen properties.

```
<jsp:setProperty name="bean id | attribute name" property="*" />
```

eg : <jsp:setProperty name="user_bean" property="*" />

WC - session.getAttribute("user_bean") - Java bean reference , invokes ALL MATCHING setters.

Request param names MUST match with Java bean's property names.

3. <jsp:getProperty name="bean id | attribute name" property="propertyName"/>

WC invokes a getter from the Java bean.

4. To invoke B.L method of java bean , use EL syntax.

eg - Suppose B.L method in Java bean

```
public String authenticateUser()
```

```
{.....}
```

How will you invoke it using EL syntax ?

```
${sessionScope.user_bean.authenticateUser()}
```

Java - session.getAttribute("user_bean").authenticateUser() -> sent to clnt

5. Navigate clnt to the next page in the SAME request(server pull : forward)

6. How to invoke getters of JB ?

OR

EL syntax :

7. Any problems observed with server pull(RD's forward scenario)

Multiple submission issue

Why ? Client is navigated to the next page , in the SAME request

Can result into increased DB traffic or even worse --dup records

eg dup orders.

What's the solution ? Double/multiple submit guard

Design Pattern : PRG (Post--Redirect--Get)

Replace server pull by client pull

(i.e replace RD's forward by sendRedirect)

There exists NO JSP standard action for redirect.

So instead of using scriptlets , use JSTL

(JSP Standard Tag library)

What is it ?

It's a part of Java EE specifications.

Implementation is left to vendors.

BUT Tomcat - doesn't supply implementation.

So needs to add , dependency

jstl1.2.jar

8. Enter JSTL

Import JSTL core tag lib n use tag.

Steps

1. Copy/confirm jstl1.2.jar under <tomcat> / lib OR <WEB-INF>/lib

(Already done in the supplied server)

2. Import JSTL supplied core tag lib

<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>

3. Use the JSTL core tags

<c:redirect url="next page URL/redirect location"/>

Performs redirect + URL encoding

(response.sendRedirect(response.encodeRedirectURL(url));

3. Use the tag

eg : In one.jsp :

<c:redirect url="two.jsp"/>

WC : response.sendRedirect(response.encodeRedirectURL("two.jsp"));

If cookies are enabled ,WC sends location : two.jsp

OR

If cookies are disabled , WC sends resp : two.jsp;jsessionid=tgret45645654hfdhgf

Other JSTL Tags from core tag library

1. Setting an attribute , under required scope

```
<c:set var="abc" value="${param.name}" scope="session"/>
```

WC : session.setAttribute("abc",request.getParameter("name"));

eg - 1. `<c:set var="abc" value="${param.name}" />`

WC : pageContext.setAttribute("abc",request.getParameter("name"));

2. Removing an attribute , from given scope

```
<c:remove var="abc" scope="session"/>
```

WC : session.removeAttribute("abc") (equivalent to Map's remove)

3. `<c:set var="abc" value="${param.name}"/>`

WC : pageContext.setAttribute("abc",request.getParameter("name"));

4. For conditional testing -

```
<c:if test="boolean value">
```

will be invoked in case of true

```
</c:if>
```

eg : In HTML form

```
<input type="submit" name="btn" value="Withdraw">
```

```
<input type="submit" name="btn" value="Deposit">
```

In transactions.jsp

```
<c:if test="${param.btn eq 'Withdraw'}">
```

In withdraw

```
</c:if>
```

WC :

```
if(request.getParameter("btn").equals("Withdraw")
```

```
out.print("in withdraw")
```

Use case : voter login --success -not yet voted --list.jsp ---status.jsp

voter login --success -alrdy voted ----status.jsp

status.jsp : after casting a vote => increment votes n change voting status

directly : u have alrdy voted => no action --display a message

eg :

```
<c:if test="${sessionScope.voter.status}">
```

You have alrdy voted....

```
</c:if>
```

5. For switch-case

```
<c:choose> Boolean
```

```
<c:when test="boolean">
```

```
</c:when>
```

```
<c:when test="boolean">
```

```
</c:when>
```

```
<c:when test="boolean">
```

```
</c:when>
```

```
<c:otherwise>
```

```
</c:otherwise>
```

```
</c:choose>
```

eg -

```
<c:choose>
```

```
<c:when test="${param.btn eq 'Withdraw'}">
```

In withdraw

```
</c:when>
<c:when test="${param.btn eq 'Deposit'}">
In Deposit
</c:when>
<c:otherwise>
some other btn .....
</c:otherwise>
</c:choose>
```

URL :
http://host:port/one.jsp?name="Ranjana"

In one.jsp
<c:set var="abc" value="\${param.name}" scope="session"/>
<c:redirect url="two.jsp"/>
WC : response.sendRedirect(response.encodeRedirectURL("two.jsp"));

In two.jsp
\${sessionScope.abc} : Ranjana ---Even if cookies are enabled/disabled

eg :
<c:redirect url="\${sessionScope.user.validateUser()}.jsp"/>

6. Iterator tag

Works as forEach | for

```
<c:forEach var="loop local attribute" items="Iterable src - List | Set | Map | array |string">
body of for each loop
</c:forEach>
```